

COS40007 Artificial Intelligence Engineering
Group Task 1: System Blueprint & Project Planning

Studio Session: Week 5 | Due: 15/04/2026

Studio Session No.	5
Group No.	CL09_T05
Project Title	Obesity Level Estimation Using Eating Habits & Physical Condition
Dataset	ObesityDataSet_raw_and_data_synthetic.csv (2,111 records, 17 features)
Group Members	Shwetha Weerasinghe - 104511483 Dewnaka Dulneth - 104477226 Kaveesha Rashvika - 104493934
Submission Date	15 April 2026

1. Group Agreement & Project Plan

1.1 Group Style and Roles

Our group of three will operate with clearly defined primary responsibilities while collaborating across all areas. The Scrum Master role rotates each sprint to distribute leadership experience. The team member responsible for submitting deliverables is the Scrum Master for that sprint. Questions to the teaching team are raised by whichever member owns the relevant sprint deliverable.

Name	Role	Responsibilities
Dewnaka	Scrum Master (Sprint 1 & 3) / ML Engineer	Sprint 1 & 3 facilitation, model training, hyperparameter tuning, evaluation, final submission coordination
Shwetha	Data Engineer / Scrum Master (Sprint 2)	Sprint 2 facilitation, data ingestion, preprocessing, EDA, feature engineering
Kaveesha	DevOps Lead / Scrum Master (Sprint 4)	Sprint 4 facilitation, GitHub management, CI/CD pipeline setup, Trello board, project documentation

1.2 Communication Plan

The group will meet twice per week throughout the project. All shared artefacts are stored in a group Google Drive folder. Teams is the primary async channel.

- Sync meeting: Every Tuesday via WhatsApp (60 min) — agenda shared the day before
- Async check-in: Every Friday via Microsoft Teams group chat
- Document sharing: Google Drive (shared folder accessible to all members)
- Response expectation: All messages acknowledged within 24 hours on weekdays
- Task tracking: Trello board — updated before each Tuesday meeting

1.3 On-Track Behaviours

- Attending all scheduled meetings or notifying the group at least 12 hours in advance
- Completing assigned sprint tasks before the agreed internal deadline (2 days before submission)
- Reviewing teammates' pull requests within 24 hours of being assigned
- Communicating blockers as soon as they are encountered rather than at the last minute
- Keeping the Trello board up to date with task progress

1.4 Off-Track Behaviours

- Missing meetings without prior notice
- Pushing incomplete or untested code to the main branch
- Not responding to messages for more than 48 hours without explanation
- Unilaterally changing agreed design or modelling decisions without group discussion

1.5 Resolving Tensions

If a conflict arises, the group will first attempt resolution through direct, respectful discussion in the Teams group chat or at the next scheduled meeting. If the issue remains unresolved after 48 hours, the current Scrum Master will document the disagreement and escalate to the unit tutor. All resolutions and key decisions are logged in the shared meeting notes file on Google Drive.

1.6 Team Expectations

#	Expectation
1	Every member contributes equally to each sprint's deliverables and reviews.
2	All code merges to main require at least one peer review via GitHub pull request.
3	All members attend the pre-submission review meeting and sign off on the final deliverable.

1.7 Sprint Plan

The project is structured across four two-week sprints aligned to the unit assessment schedule, following Agile/Scrum methodology.

Sprint	Dates (Approx.)	Goals & Deliverables	Owner
Sprint 1 (Current)	Wk 7 - 8	System blueprint diagram, group agreement, project setup (GitHub repo, Trello board, Google Drive, Notion docs). Deliverable: Task 1 submission.	Dewnaka
Sprint 2	Wk 9 - 10	Data ingestion pipeline, EDA report, preprocessing (missing values, encoding, normalisation), feature selection. Internal deliverable: clean dataset + EDA notebook.	Shwetha
Sprint 3	Wk 11	Baseline model (Logistic Regression), primary model (Random Forest / XGBoost), evaluation (accuracy, F1-macro, confusion matrix), MLflow experiment tracking.	Dewnaka
Sprint 4	Wk 12	Streamlit web interface, GitHub Actions CI/CD pipeline, monitoring setup, final report, demo preparation. Deliverable: Final submission.	Kaveesha

2. System Blueprint Diagram

The diagram below provides a high-level architecture of the Obesity Level Estimation System. Raw data from the UCI dataset flows through a preprocessing and feature engineering pipeline before being consumed by a multiclass classification model. Predictions (one of 7 obesity level classes) are presented through a Streamlit web interface targeted at health professionals or researchers.

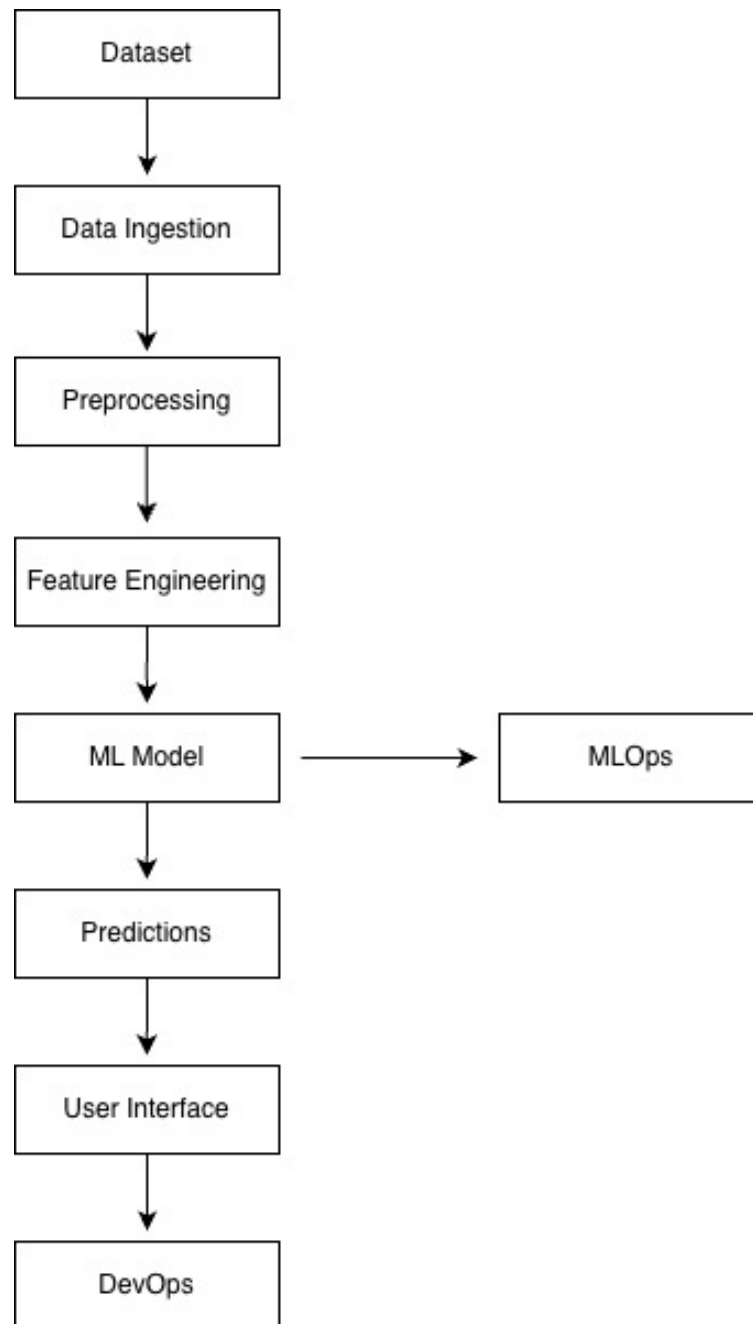


Figure 1: High-level system architecture showing data flow across all pipeline stages.

Component Descriptions

Component	Description	Technology	Sprint
Data Source	UCI Obesity Dataset: 2,111 records, 16 input features, 1 target (NObeyesdad — 7 classes). Mix of real and synthetic data.	CSV file	Sprint 1
Data Ingestion	Load raw CSV into a Pandas DataFrame. Validate schema, check for nulls and unexpected values. Log data shape and types.	Python, Pandas	Sprint 2
Preprocessing	Encode categorical features (LabelEncoder / OneHotEncoder). Normalise numeric columns (MinMaxScaler). Handle any class imbalance via SMOTE if needed.	Scikit-learn, imbalanced-learn	Sprint 2
Feature Engineering	Derive BMI from Height/Weight. Evaluate feature importance via Random Forest permutation. Drop low-importance features.	Scikit-learn, Pandas	Sprint 2–3
ML Model	Primary: Random Forest Classifier or XGBoost (multiclass). Baseline: Logistic Regression. Tuned via GridSearchCV.	Scikit-learn, XGBoost	Sprint 3
Model Output	Predicted obesity class (one of 7 levels) + class probabilities. Evaluation: accuracy, F1-macro, confusion matrix.	Scikit-learn metrics	Sprint 3
User Interface	Streamlit web app where a user enters personal health data and receives a predicted obesity level with probability breakdown.	Streamlit, Python	Sprint 4
MLflow Tracking	Log all training runs: parameters, metrics, model artefacts. Model Registry for versioning.	MLflow	Sprint 3–4

3. MLOps & DevOps Integration (High-Level Design)

3.1 Data & Training Strategy

The dataset (2,111 records) will be split using a stratified random approach to preserve proportional representation of all 7 obesity classes across each subset. Stratification is essential here because some classes (e.g., Obesity_Type_III) may have fewer samples.

Split	Proportion	Purpose
Training set	70% (~1,478 records)	Model fitting, cross-validation (5-fold StratifiedKfold), hyperparameter tuning
Validation set	15% (~317 records)	Early stopping, model selection between algorithms
Test set	15% (~316 records)	Final unbiased evaluation — held out until the last sprint; never used during development

As this is a static dataset (no live data feed), new data incorporation is not applicable in the initial version. However, the pipeline is designed to accept new CSV batches so that if updated survey data becomes available, it can be incorporated with minimal changes. In that scenario, the test set would be refreshed to include records from the new cohort.

3.2 Retraining Strategy

Given the static nature of the dataset in this project, retraining will be triggered by the following conditions:

- Performance-based: If the model's F1-macro score on the validation window drops below 0.80, or accuracy drops below 82%, retraining is triggered. This would apply if new data is introduced.
- Time-based (future consideration): In a production health system, retraining would be scheduled quarterly to incorporate new patient survey data and account for population-level dietary shifts.
- Manual trigger: Any significant change in feature engineering logic or preprocessing steps will trigger a full retraining run, logged in MLflow.

Retraining will be scripted as a standalone Python module (retrain.py) that can be invoked manually or via a GitHub Actions scheduled workflow. All retraining runs are versioned in MLflow.

3.3 Monitoring & Drift Detection

Model Performance Monitoring

After the model is deployed within the Streamlit app, predictions will be periodically reviewed against any available ground truth. Key metrics to monitor: accuracy, F1-macro (overall), per-class F1-score (important for detecting degradation on minority classes like Obesity_Type_III), and AUC-ROC (one-vs-rest).

Data Drift

Data drift will be assessed by comparing the statistical distribution of incoming input features against the training distribution baseline. For numeric features (Age, Height, Weight, BMI), we will monitor mean and standard deviation shifts. For categorical features (Gender, CAEC, CALC, MTRANS), chi-squared tests will flag distributional changes. A Population Stability Index (PSI) value above 0.2 on any key feature will trigger a data quality alert and prompt a review of the preprocessing pipeline.

Concept Drift

Concept drift — where the relationship between features and obesity class changes over time (e.g., due to population-level lifestyle changes) — will be detected by monitoring model performance on rolling windows of incoming predictions. A statistically significant drop in F1-macro over two consecutive rolling windows will flag a potential concept drift event, prompting investigation and likely retraining with refreshed data.

3.4 DevOps Practices

Version Control — GitHub

All project code will be managed on a shared GitHub repository using a feature-branch workflow. The repository will be structured as follows:

- Main branch: stable, submission-ready code only. Direct pushes to main are not permitted.
- Feature branches: named by task (e.g., feature/data-pipeline, feature/xgboost-model, feature/streamlit-ui).
- Pull requests: every merge to main requires at least one peer review and passing CI checks.
- Commit convention: conventional commits format (feat:, fix:, docs:, refactor:, test:).
- GitHub Issues: used to track bugs, tasks, and improvements linked to Trello cards.

CI/CD Pipeline — GitHub Actions (Conceptual Design)

Full CI/CD implementation is planned for Sprint 4. The conceptual pipeline is designed as three stages:

Stage	Trigger	Actions
CI — Lint & Test	Every pull request to main	Run flake8 linting, pytest unit tests on preprocessing functions, data schema validation checks
CD — Model Build	Merge to main	Execute training pipeline, log run to MLflow, save model artefact, version and register in MLflow Model Registry
CD — Deploy	Release tag (e.g., v1.0.0)	Deploy updated Streamlit app to hosting environment (e.g., Streamlit Cloud or local server)

Code and Model Versioning

- Code versioning: GitHub semantic tags (v1.0.0, v1.1.0) marking each sprint's stable release.
- Model versioning: MLflow Model Registry with Staging and Production lifecycle stages. Each registered model version includes the training dataset snapshot hash, hyperparameters, and evaluation metrics.
- Data versioning: Raw CSV snapshots are stored with timestamps in /data/raw/. Processed datasets are stored separately in /data/processed/ with version suffixes.
- Notebook versioning: Jupyter notebooks tracked in GitHub; outputs cleared before commits to avoid large diffs.

4. Project Management Setup (Agile Tools)

4.1 User Stories

The following user stories define the functional requirements of the system and were used to drive backlog creation and sprint planning:

ID	User Story	Priority
US-01	As a health researcher, I want to input a person's eating habits and physical data so that I can receive a predicted obesity level classification.	High
US-02	As a data engineer, I want to automate loading and validating the dataset so that I can ensure data quality before model training.	High
US-03	As an ML engineer, I want to compare at least two classification algorithms so that I can select the best-performing model for obesity level prediction.	High
US-04	As a health researcher, I want to see the probability breakdown across all 7 obesity classes so that I can understand prediction confidence.	Medium
US-05	As a system administrator, I want to be alerted when model F1-score drops below a threshold so that I can trigger retraining before accuracy degrades.	Medium
US-06	As an end user, I want the system to clearly explain which features most influenced the prediction so that I can understand the result.	Medium

4.2 Backlog & Task Breakdown

Story	Task	Assigned To	Sprint
US-01	Design Streamlit input form (16 feature fields)	Kaveesha	Sprint 1 (wireframe), Sprint 4 (build)
US-02	Write CSV ingestion + schema validation script	Shwetha	Sprint 2
US-02	Implement preprocessing pipeline (encoding, scaling)	Shwetha	Sprint 2
US-02	Write pytest unit tests for preprocessing functions	Shwetha	Sprint 2
US-03	Train Logistic Regression baseline + evaluate	Dewnaka	Sprint 3
US-03	Train Random Forest / XGBoost + GridSearchCV tuning	Dewnaka	Sprint 3
US-03	Log all experiments to MLflow, compare F1-macro	Dewnaka	Sprint 3

US-04	Add probability bar chart to Streamlit dashboard	Kaveesha	Sprint 4
US-05	Configure MLflow metric threshold alerts in GitHub Actions	Kaveesha	Sprint 4
US-06	Compute and display SHAP feature importance in dashboard	Dewnaka	Sprint 4

4.3 Project Management Tool — Microsoft Teams

The group uses Microsoft Teams for sprint task management and collaboration. Tasks are managed using the Planner (Tasks by Planner and To Do) feature within Teams. The board is organised into five columns: Backlog, Sprint In Progress, Review, Done, and Blocked. Each task corresponds to an item from Section 4.2, labelled by sprint and assigned to a team member.

- Microsoft Teams Link: [Insert Teams Channel Link] *(replace with actual link before submission)*
- Task labels: Sprint 1 (blue), Sprint 2 (green), Sprint 3 (orange), Sprint 4 (red), Bug (purple)
- Each task includes: task description, assigned member, due date, and checklist of sub-tasks

4.4 Project Documentation

All project documentation is maintained in a shared Notion workspace accessible to all group members. The workspace contains:

- Project description and scope statement
- System overview and key design decisions log
- Meeting notes and action items (updated after every Tuesday meeting)
- MLflow experiment log summary
- References and dataset citation

5. Contribution Statement

All group members contributed equally to this task. The table below reflects primary ownership of each deliverable component:

Name (Student ID)	Primary Contributions	Approx. %
Dewnaka (104477226)	System blueprint component descriptions, MLOps design (retraining strategy, monitoring, drift detection), model component planning, ML experiment strategy.	33%
Shwetha (104511483)	Data & training strategy, dataset analysis, preprocessing design, user stories US-01 to US-03, backlog task breakdown, EDA planning.	33%
Kaveesha (104493934)	Group agreement document, communication plan, sprint plan, Trello board setup, Notion documentation, DevOps design (GitHub strategy, CI/CD pipeline concept), document formatting.	34%

All group members reviewed and approved this submission prior to upload on 15 April 2026.

6. Group GitHub Repository

All project code, notebooks, and version history are maintained in the group GitHub repository:

https://github.com/COS40007-2026-Classrooms/CL09_T05

Repository structure planned for Sprint 1:

- /data/raw/ — original CSV dataset
- /data/processed/ — cleaned and encoded datasets
- /notebooks/ — EDA and modelling Jupyter notebooks
- /src/ — production Python scripts (ingestion, preprocessing, training, app)
- /tests/ — pytest unit tests
- /.github/workflows/ — GitHub Actions CI/CD pipeline YAMLS
- /models/ — saved model artefacts (MLflow linked)
- README.md — project overview, setup instructions, team info